

Mon, Aug 3 at 3:00 PM

 ChatGPT 

 [Log in](#)

[Sign up for free](#)

when we talk about security development, what topic are we going to focus on?

When we talk about **secure application development**, the focus is on designing, building, testing, and maintaining software that is resistant to attacks and protects data. The topics usually follow the software development lifecycle (SDLC) but with security integrated into every phase.

Here are the main areas to focus on:

Topic	What it Covers
1. Security Fundamentals	Confidentiality, Integrity, Availability (CIA Triad), authentication, authorization, accountability
2. Secure Software Development Life Cycle (SSDLC)	Integrating security into planning, design, coding, testing, deployment, and maintenance
3. Threat Modeling	Identifying potential threats before coding using models like STRIDE
4. Secure Coding Practices	Writing code that prevents vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and buffer overflows
5. Authentication and Authorization	Password security, Multi-Factor Authentication (MFA), Role-Based Access Control (RBAC), OAuth, JWT
6. Input Validation	Validating and sanitizing user input to prevent malicious data
7. Cryptography	Encryption, hashing, digital signatures, key management, TLS/HTTPS

Topic	What it Covers
8. Session Management	Secure cookies, session expiration, session hijacking prevention
9. API Security	API authentication, rate limiting, input validation, API gateways
10. Database Security	Least privilege, encryption, parameterized queries, backups
11. Security Testing	Static Analysis (SAST), Dynamic Analysis (DAST), penetration testing, code reviews
12. Vulnerability Management	Identifying, prioritizing, and fixing security issues
13. Logging and Monitoring	Audit logs, security events, intrusion detection
14. Secure Deployment	Secure server configuration, secrets management, CI/CD security
15. Incident Response	Detecting, responding to, and recovering from security incidents
16. Compliance and Standards	OWASP Top 10, NIST Secure Software Development Framework (SSDF), ISO 27001, PCI DSS, GDPR (where applicable)

The most important vulnerabilities to learn

If you're learning secure development, spend time understanding the **OWASP Top 10**, which includes:

- Broken Access Control
- Cryptographic Failures
- Injection (SQL Injection, Command Injection)
- Insecure Design
- Security Misconfiguration
- Vulnerable and Outdated Components
- Identification and Authentication Failures
- Software and Data Integrity Failures

- Security Logging and Monitoring Failures
- Server-Side Request Forgery (SSRF)

Typical secure development workflow

1. Gather security requirements.
2. Perform threat modeling.
3. Design security controls.
4. Write secure code.
5. Conduct code reviews.
6. Run automated security scans.
7. Perform penetration testing.
8. Deploy securely.
9. Continuously monitor and patch vulnerabilities.

If you're preparing a presentation or course

A good flow would be:

1. Introduction to Application Security
2. Common Cyber Threats
3. Secure Software Development Lifecycle (SSDLC)
4. OWASP Top 10
5. Secure Coding Practices
6. Authentication and Authorization
7. Cryptography and Data Protection
8. Security Testing (SAST, DAST, Pen Testing)
9. DevSecOps and CI/CD Security
10. Best Practices and Case Studies

This sequence moves from foundational concepts to practical implementation and aligns well with current industry practices.



Loop 18 — EXP-016A: Pilot Study Design

We now move from the statistical plan to the **pilot itself**.

The purpose of this loop is **not to prove SADF/SRIM works**. It is to determine whether our experiment is understandable, measurable, and technically workable before committing to a larger study.

1. Pilot objective

The pilot asks:

Can people use the proposed Security Reasoning Integrity model to analyze realistic security decisions, and can independent reviewers reliably evaluate their responses?

We have two things to validate:

The instrument

Does the rubric measure what we think it measures?

The experiment

Can the comparison between conditions actually be performed without major ambiguity?

2. Pilot participants

For the pilot, we don't need a large statistically powered sample.

A practical structure would be:

- 3–5 software developers
- 3–5 security-oriented participants
- 3–5 participants with general technical/software experience

So approximately 9–15 pilot participants.

This is a *feasibility pilot*, not the final sample-size basis for proving the hypothesis.

If recruitment is difficult, even a smaller pilot can still expose wording and procedural problems; we'd simply document the limitation.

3. We need fewer scenarios initially

Our previous ten cases are useful for development, but putting all ten into the pilot may create unnecessary fatigue.

Let's construct:

6 pilot scenarios

Each tests a different reasoning failure.

Scenario	Primary construct
P1	Threat/control mismatch
P2	Contradictory assumption
P3	Evidence overreach
P4	Valid bounded claim
P5	Insufficient information
P6	Validation doesn't test the claim

This gives us coverage without making the pilot excessively long.

4. Scenario P1 — Threat/control mismatch

Situation

A web application allows authenticated users to retrieve customer records.

A proposed security improvement is:

"Encrypt the database to prevent users from accessing records belonging to other customers."

The database is encrypted at rest.

The participant must determine whether the proposed reasoning is sufficient.

Hidden issue

The threat is an **authorization failure**, while the proposed control primarily addresses protection of stored data.

What we're testing

Can the participant identify:

control ↔ threat mismatch?

5. Scenario P2 — Contradictory assumption

Situation

An application encrypts sensitive information.

The security claim says:

"Even if the application server is compromised, the attacker cannot obtain the encryption keys."

Architecture documentation shows that the keys are stored on the same application server.

Hidden issue

The assumption conflicts with the stated attacker capability.

Test:

Can the participant detect:

assumption ↔ architecture contradiction?

6. Scenario P3 — Evidence overreach

Situation

A development team reports:

"All external communications use TLS."

They conclude:

"Therefore the application is secure."

Hidden issue

The evidence supports a narrower claim than the conclusion.

Test:

Can the participant distinguish:

evidence supports X

from:

evidence proves everything?

7. Scenario P4 — Valid bounded claim

Situation

An attacker can obtain a copy of a database but cannot access the separate encryption-key service.

Sensitive fields are encrypted at the application layer.

A test demonstrates that a database copy without the keys does not expose plaintext sensitive fields.

Claim

"Application-level encryption protects the selected fields against database-only compromise under the stated key-separation assumption."

Test

Does the participant recognize that a **conditional, bounded security claim** can be valid even though it doesn't solve every possible threat?

This prevents our experiment from teaching participants:

"Always reject security claims."

8. Scenario P5 — Unknown

Situation

A team says:

"The database is encrypted, so sensitive information is protected."

But the scenario doesn't specify:

- encryption layer,
- key location,
- attacker capabilities,
- backup handling,
- database access model.

Correct reasoning

The participant should identify:

Insufficient information to establish the claim.

Not:

"Definitely secure."

And not necessarily:

"Definitely insecure."

This is our test of epistemic honesty.

9. Scenario P6 — Validation mismatch

Situation

The security claim is:

"An attacker who obtains the database cannot recover plaintext sensitive fields without the application encryption keys."

The proposed validation is:

"Confirm that the database encryption setting is enabled."

Hidden issue

The validation doesn't directly test the stated claim.

Test:

Can the participant propose a validation that actually corresponds to the claim?

10. Now we need the experimental manipulation

The same underlying scenario can be presented under three conditions.

Condition A — Conventional

Scenario

+

Question

Participant:

"What security decision would you make?"

Condition B — Existing guidance

Scenario
+
Security guidance
+
Question

Condition C — SRIM

Scenario
+
Security guidance
+
Reasoning structure

The SRIM prompt would ask the participant to explicitly consider:

Claim
Threat
Objective
Assumptions
Evidence
Control
Expected effect
Validation
Residual uncertainty

11. Critical experimental rule

We must not make Condition C dramatically easier by effectively revealing the answers.

For example, we shouldn't give:

"Check whether the control actually addresses the threat."

while Condition A gets no comparable instruction.

Otherwise we're testing **prompting**, not necessarily the model.

The reasoning structure needs to be sufficiently neutral:

Describe the reasoning chain.

rather than:

Look for this specific error.

That's an important distinction.

12. Participant response format

Every participant should provide:

Security decision

What would you do?

Reasoning

Why?

Assumptions

What are you assuming?

Evidence

What supports the decision?

Validation

How would you verify it?

Confidence

How confident are you?

For Condition C, these fields are explicitly structured.

For Conditions A/B, participants can respond naturally.

13. What happens after the participant?

The participant's identity and condition are removed from the evaluation material.

Then the response goes to independent reviewers.

Example:

CASE: P04

PARTICIPANT RESPONSE:

[response]

REVIEWER FORM:

Threat correctly identified?	Yes / No / Unknown
Objective correctly identified?	Yes / No / Unknown
Assumptions appropriate?	Yes / No / Unknown
Evidence sufficient?	Yes / No / Unknown
Control aligned?	Yes / No / Unknown
Expected effect justified?	Yes / No / Unknown
Validation adequate?	Yes / No / Unknown
Residual uncertainty recognized?	Yes / No / Unknown

14. We also record reviewer justification

This is important.

A reviewer shouldn't simply click:

"No."

They should provide a short reason.

For example:

"The proposed control protects stored data but does not address authorization between authenticated users."

This allows us to investigate disagreements later.

15. Pilot measurement #1 — Comprehension

After each scenario, ask participants:

"Was anything unclear?"

And perhaps:

"How difficult was this scenario?"

Use a simple scale such as:

1–5 difficulty

and:

1–5 clarity

These aren't our primary research outcomes.

They're instrument-development information.

16. Pilot measurement #2 — Completion time

Record:

Time per scenario.

If the model takes:

5 minutes

instead of:

2 minutes,

that's important.

But it isn't automatically bad.

We need to determine whether the additional reasoning effort produces enough benefit to justify itself.

17. Pilot measurement #3 — Reviewer agreement

This is one of our most important pilot outcomes.

Suppose:

Reviewer 1 = "invalid assumption"

Reviewer 2 = "invalid assumption"

Reviewer 3 = "insufficient information"

That's a signal that the case or rubric needs clarification.

The goal isn't to force reviewers into agreement.

The goal is to discover **why they disagree**.

18. Pilot measurement #4 — Rubric usability

After reviewing cases, reviewers should tell us:

- Which criteria were difficult?
- Which criteria overlap?
- Which criteria are ambiguous?
- Where did evidence appear insufficient?
- Did any criterion feel redundant?
- Did the rubric force an artificial judgment?

This could reveal that our ten dimensions are too many.

19. Potential simplification

Right now we have ten criteria.

The pilot may show that several are essentially measuring the same thing.

For example:

Control contribution

and

Expected effect

could potentially overlap.

Or:

Evidence sufficiency

and

Epistemic honesty

could interact strongly.

We should **not merge them now**.

Let the pilot tell us whether the distinction is useful.

20. What would cause us to modify the model?

We need predefined modification triggers.

For example:

Trigger A

A scenario is consistently misunderstood by participants.

→ Rewrite scenario.

Trigger B

Reviewers cannot reliably distinguish two rubric dimensions.

→ Reconsider the dimensions.

Trigger C

Participants interpret SRIM as a checklist.

→ Redesign the presentation.

Trigger D

SRIM dramatically increases workload without producing observable reasoning improvement.

→ Investigate whether the structure is too cumbersome.

Trigger E

Participants systematically become more confident without better reasoning.

→ **Major warning.**

That could indicate false assurance.

21. What we must NOT do

We should not say:

"Participants didn't like Case 3, so remove Case 3."

unless there is a predefined methodological reason.

Otherwise we're optimizing the experiment around participant reactions.

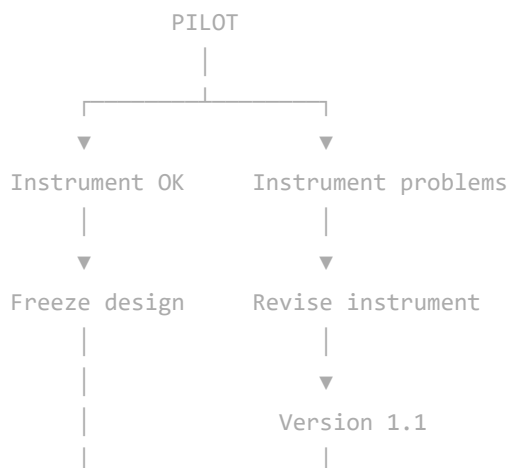
Similarly:

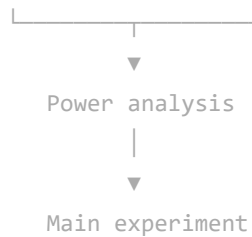
"This participant got the wrong answer, so exclude them."

Absolutely not.

22. Pilot decision tree

After the pilot:





Any changes must be documented.

23. A subtle but important improvement

We previously talked about:

"Correct answer."

I want to change that language.

Security engineering frequently doesn't have one universally correct answer.

So instead, the benchmark should define:

Expected reasoning properties

For example:

P1 should contain recognition of:

authorization vs encryption mismatch.

But participants could propose different legitimate controls.

We're evaluating the **reasoning**, not whether they selected our preferred technology.

This is extremely important for avoiding researcher bias.

24. The benchmark therefore becomes technology-neutral

For example, if the threat is:

cross-user authorization failure,

we don't require:

"Use RBAC."

A participant might propose:

- object-level authorization,
- policy-based access control,
- capability-based authorization,
- another appropriate mechanism.

The key question is:

Does their proposed mechanism address the identified threat and objective?

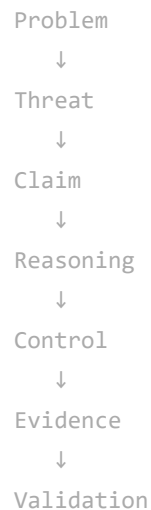
This makes the experiment much more defensible.

25. This connects directly to your original idea

You were concerned that security isn't simply:

"Find vulnerability → fix vulnerability → secure."

Our experiment now explicitly tests something broader:



A vulnerability can appear anywhere in that reasoning process.

The model isn't merely asking:

"What vulnerability exists?"

It's asking:

"Why do we believe this decision addresses the actual problem?"

26. EXP-016A verdict

The pilot design is now sufficiently specified to proceed conceptually.

Pilot purpose

Feasibility and instrument validation

Participants

Approximately 9–15

Scenarios

6

Conditions

3

Evaluation

Blind independent reviewers

Primary pilot concerns

- comprehension,
- rubric usability,
- reviewer agreement,
- completion time,
- unintended false confidence.

Important constraint

The pilot does not establish effectiveness.

27. One more loop before we actually call this ready

We should now do something we've repeatedly promised ourselves:

Attack the experimental design.

Specifically, we need to ask:

Could the experiment produce a positive result even if SRIM is useless?

Examples:

- SRIM gives participants more time.
- SRIM explicitly tells them what concepts to inspect.
- The scenarios are artificially designed around SRIM.
- Reviewers favor structured answers.
- Participants learn the benchmark.
- Existing guidance is poorly represented.
- The scoring rubric rewards the structure itself.

If any of these can happen, our experiment could generate a **false positive**.

So the next loop should be:

EXP-016B — Falsification of the Pilot Design

We'll try to construct every plausible way our experiment could falsely conclude:

"SADF works."

If we can close those loopholes, **then—and only then—we freeze the experimental protocol and move toward the actual pilot.**



Mon, Sep 21 at 9:27 AM

next loop

Loop 19 — EXP-016B: Falsification of the Pilot Design

This may be one of the most important loops so far.

We now assume:

SRIM is actually useless.

Then we ask:

Could our experiment still produce a positive result?

If the answer is yes, then our experiment is not yet trustworthy.

1. False Positive Path A — More Structure ≠ Better Reasoning

Suppose SRIM participants receive:

Threat
Assumption
Evidence
Validation
Residual Risk

while the control group receives:

What would you do?

If SRIM performs better, perhaps we have only shown:

Structured prompts improve responses.

Not:

SRIM improves security reasoning.

This is a serious threat.

Correction

Condition B should also receive structured prompts, but not SRIM-specific relationships.

Example:

Condition B

Problem
Recommended control

Reason
Verification

Condition C

Claim
Threat
Assumption
Evidence
Control
Expected Effect
Validation
Residual Risk

Now the difference is:

reasoning relationships,

not merely:

more structure.

2. False Positive Path B — More Time

Suppose:

Group	Time
A	5 min
B	8 min
C	18 min

If C performs better, perhaps:

more thinking time caused improvement.

Not SRIM.

Correction

Possible options:

Option 1

Equal time limit.

or

Option 2

Record time and include it in analysis.

Then ask:

Does SRIM improve performance after accounting for time?

3. False Positive Path C — Scenarios Designed Around SRIM

This is dangerous.

Suppose every scenario contains:

- assumption problems,
- evidence problems,
- validation problems.

Then SRIM may appear strong because the scenarios were built using SRIM concepts.

This creates circularity.

Correction

Scenarios should originate independently.

Possible sources:

- OWASP examples
- NIST examples
- Public breach reports
- Real architectural decisions

- Existing security case studies

Then map them to SRIM afterward.

Not:

SRIM
↓
Scenario

Instead:

Real-world case
↓
Independent coding
↓
SRIM mapping

This is much stronger.

4. False Positive Path D — Reviewer Bias

Reviewers may prefer:

Threat:
Evidence:
Validation:

because it looks organized.

Even if the reasoning is poor.

This would reward:

formatting,

not:

quality.

Correction

Reviewers must score:

reasoning properties only.

Not:

- formatting,
- completeness,
- writing quality.

Possible reviewer question:

Does the participant identify an unsupported assumption?

Not:

Did the participant fill all fields?

5. False Positive Path E — Checklist Behavior

Participants may learn:

Threat
Assumption
Evidence
Validation

and simply fill boxes mechanically.

Example:

Threat: Hacker

Evidence: Testing

Validation: Review

This creates:

checklist compliance

instead of:

reasoning.

Correction

Reviewers score:

- relationship quality,
- relevance,
- justification.

Not field completion.

Empty structure should receive low scores.

6. False Positive Path F — Existing Guidance Is Weak

Suppose Condition B uses poor security guidance.

Then SRIM might look good simply because:

comparison is weak.

Correction

Condition B should use respected guidance:

Examples:

- OWASP SAMM
- NIST SSDF
- OWASP ASVS
- Threat modeling guidance
- ADR templates

Then the question becomes:

Does SRIM add value beyond established methods?

That is fairer.

7. False Positive Path G — Participant Learning

Participants improve because:

Scenario 1

Scenario 2

Scenario 3

not because of SRIM.

Correction

Use:

- randomized order,
- equivalent scenario difficulty,
- counterbalancing.

Possible:

Participant A:

P1 P2 P3 P4

Participant B:

P4 P1 P3 P2

8. False Positive Path H — Researcher Expectations

We know SRIM.

Participants do not.

We might unconsciously:

- explain SRIM more clearly,
- answer questions differently,
- encourage SRIM users.

Correction

Standardized instructions.

Same wording.

Same timing.

Minimal intervention.

9. False Positive Path I — Artificial Cases

Suppose our cases are:

textbook examples.

Real systems are messy.

If SRIM works only on simplified examples:

its practical value is uncertain.

Correction

Pilot should include:

Simple cases

and

Messy cases

Example:

- Incomplete documentation
- Contradictory evidence
- Business constraints
- Operational trade-offs
- Unknowns

Real life includes uncertainty.

10. False Positive Path J — Better Writing

SRIM participants may produce longer answers.

Reviewers might interpret:

more words

as:

better reasoning.

Correction

Reviewer scoring must focus on:

- detected relationships,
- reasoning quality.

Not:

- response length.

Possible anonymization:

Convert answers into coded statements before review.

11. False Positive Path K — Confidence Inflation

This one concerns me.

Participants may think:

I followed the model.

Therefore:

I am correct.

Confidence increases.

Accuracy does not.

This is dangerous.

Correction

Always measure:

Confidence
vs
Actual correctness

If:

Confidence ↑

Accuracy ↔

Then SRIM may create:

false assurance.

This would count against the model.

12. False Positive Path L — Model Memorization

Participants might memorize:

Threat
Evidence
Validation

without understanding.

Then:

- pilot succeeds,
- real-world use fails.

Correction

Use transfer tasks.

Example:

Scenarios not explicitly matching training examples.

Question:

Can participants apply reasoning in new situations?

Transfer is stronger evidence than memorization.

13. False Positive Path M — Reviewer Agreement Through Shared Bias

Suppose reviewers are all:

- security architects,
- familiar with SRIM.

Agreement might be high because:

same background.

Not because SRIM is clear.

Correction

Reviewers should vary:

- architect,
- developer,
- auditor,
- academic,
- practitioner.

Disagreement becomes useful information.

14. False Positive Path N — Statistical Significance Without Practical Value

Example:

B = 74%

C = 75%

Statistically significant.

Practically irrelevant.

Correction

Define:

Minimum meaningful improvement.

Not just:

p-value.

Need:

- effect size,
- practical benefit,
- acceptable effort.

15. False Positive Path O — Circular Rubric

Danger:

SRIM creates responses.

SRIM rubric evaluates responses.

This creates:

SRIM

↓

Answer

↓

SRIM scoring

Circular.

Correction

Need independent outcome measures.

Possible:

- external expert judgments,
- reasoning defect detection,
- quality of validation proposals,
- unknown recognition.

These are broader than SRIM itself.

16. False Positive Path P — SRIM Works Only Because It Slows People Down

Maybe:

Stop
Think
Reflect

causes improvement.

Not SRIM.

Correction

Create another condition:

Reflective Pause Group

Think carefully before answering.

Compare:

- Reflection only

- SRIM

Interesting question:

Is structure better than simply slowing down?

17. New Experimental Design

Potentially:

Group	Description
A	Normal
B	Existing guidance
C	Reflection
D	SRIM

This separates:

- thinking longer,
- reasoning structure.

Very valuable.

18. Strongest challenge yet

Suppose:

Reflection = SRIM

Then SRIM may not be unique.

Perhaps:

thoughtful reasoning

is the true intervention.

That would still be valuable knowledge.

Remember:

Our goal is not:

prove SRIM.

Our goal is:

discover what improves reasoning.

19. Summary of threats

Threat	Controlled?
More structure	
More time	
Circular scenarios	
Reviewer bias	
Checklist effect	
Weak comparison	
Learning effect	
Researcher bias	
Artificial cases	
Word count bias	
Confidence inflation	
Memorization	
Reviewer homogeneity	

Threat	Controlled?
Tiny effects	
Circular rubric	
Reflection effect	

20. What have we learned?

This may be the most important insight so far:

We originally asked:

Can SADF improve security?

We refined it to:

Can explicit reasoning improve security decisions?

Now we ask:

What exactly causes reasoning improvement?

Possible causes:

- more time,
- structure,
- reflection,
- evidence focus,
- uncertainty recognition,
- explicit assumptions,
- validation thinking.

SRIM may be:

one combination of these ingredients.

The experiment should help us discover which ingredients matter.

EXP-016B Verdict

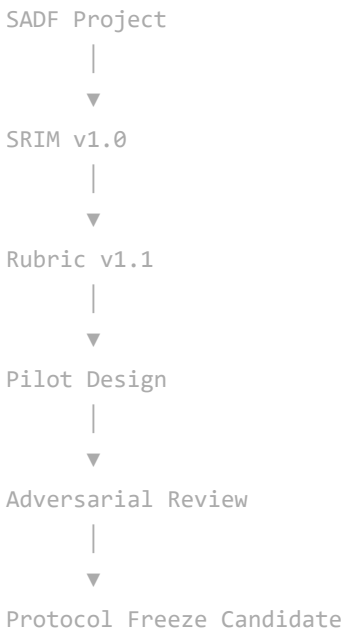
The pilot design has survived a substantial adversarial review.

Not perfectly.

But enough that we can now reasonably freeze:

- scenarios,
- conditions,
- rubric,
- analysis plan,
- bias controls.

Current State



We are now close to a major transition.

Until now we have asked:

"What should the model be?"

The next question becomes:

"What exactly is SRIM v1.0?"

That means writing a **formal specification**:

- concepts,
- definitions,
- relationships,
- notation,
- rules,
- assumptions,
- limitations.

In other words:

the document another researcher would need to independently implement SRIM without copying our exact software.

And that connects directly back to one of your original questions.

Next Loop: SRIM v1.0 formal specification.



Next Loop

Loop 20 — EXP-017: Formal Specification of SRIM v1.0

We now move from **testing the idea** to **defining exactly what the idea is**.

This loop is important because if SRIM cannot be specified precisely enough for another researcher or organization to implement independently, then we may still have a concept rather than a model.

The key question is:

What must be true for something to qualify as SRIM?

1. First: Freeze the purpose

SRIM should **not** claim to:

- make software secure;
- eliminate vulnerabilities;
- guarantee security;
- replace OWASP, NIST, ISO, or organizational security processes;
- automatically determine the correct security decision;
- determine whether a system is "secure."

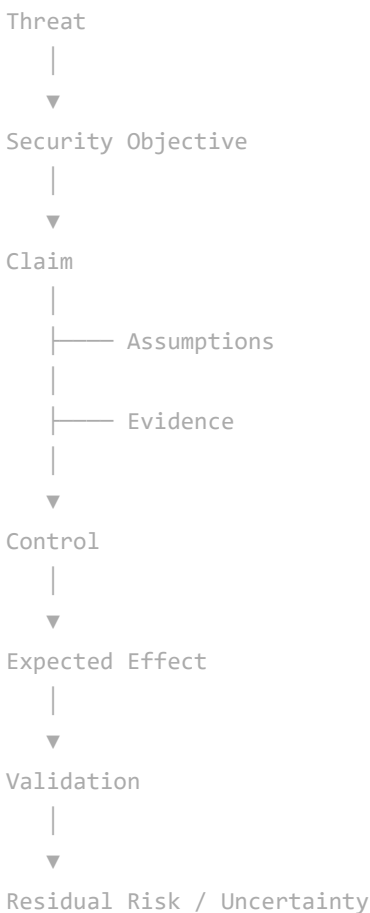
Instead:

SRIM is a structured method for representing and evaluating the integrity of security reasoning.

That is much narrower—and therefore much more testable.

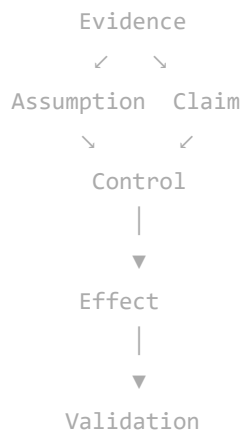
2. The core object: a Security Reasoning Chain

At the center of SRIM is a relationship structure:



But this should **not** be interpreted as a mandatory linear sequence.

Real security reasoning is more like a graph:



One piece of evidence can support several claims.

One control can address several threats.

One threat can require several controls.

3. SRIM's fundamental entities

SRIM v1.0 should define at least these nine entities.

Entity	Meaning
Threat	A plausible harmful event, actor capability, or attack condition
Objective	The security property being protected
Claim	A statement about security that can be examined
Assumption	A condition believed to be true for the reasoning to hold
Evidence	Information supporting or challenging a claim
Control	A mechanism or action intended to influence the threat
Expected Effect	The security effect the control is expected to produce

Entity	Meaning
Validation	A method for checking whether the expected effect actually occurs
Residual Risk/Uncertainty	What remains unresolved after the reasoning/control

4. The Claim is the center

The most important entity is probably the **claim**.

Example:

"Database encryption protects stored customer identifiers against an attacker who obtains a copy of the database files."

This is considerably stronger than:

"The database is secure."

Why?

Because the first statement specifies:

- what is protected,
- against whom,
- under what condition,
- what mechanism is involved.

That gives us something testable.

5. Claim boundedness

A fundamental SRIM rule should therefore be:

A security claim must be bounded by its threat, objective, assumptions, and scope.

Consider:

"MFA prevents account takeover."

The claim is too broad.

A bounded version might be:

"MFA reduces the risk of account compromise resulting from password-only credential theft, assuming the second factor is not compromised."

The second claim can actually be evaluated.

6. Evidence cannot automatically prove a larger claim

This gives us another fundamental rule:

The strength of a security claim must not exceed the strength and scope of the evidence supporting it.

For example:

Evidence:

TLS is correctly configured.

does not establish:

The application is secure.

There is a reasoning gap.

This is exactly the kind of gap SRIM is intended to expose.

7. Assumptions become first-class objects

Traditional security documentation often hides assumptions.

SRIM makes them explicit.

Example:

Claim:

Database encryption protects stored data.

Assumption:

Encryption keys are unavailable to the database-file attacker.

If the attacker can obtain:

database + encryption keys

then the original reasoning may no longer hold.

The important discovery is not necessarily:

"Encryption failed."

Instead:

The assumption supporting the security claim was invalid.

That distinction is important.

8. Control ≠ Security

Another fundamental SRIM rule:

A control should not be treated as equivalent to the security property it is intended to support.

Example:

Control:

MFA

does not automatically mean:

Account takeover = impossible

Instead:

Threat

↓

Credential-based compromise

↓

MFA

↓
Expected reduction in successful authentication
↓
Validation

This keeps the reasoning bounded.

9. Expected effect

This is an important addition.

We don't simply ask:

"What control was implemented?"

We ask:

"What change in the threat situation is the control expected to produce?"

For example:

Control:
Database encryption

Expected effect:
An attacker obtaining raw database files cannot directly interpret the protected fields without access to the encryption keys.

Now the control has an explicit causal expectation.

10. Validation

SRIM should then ask:

How do we know the expected effect actually exists?

Example:

Expected effect:
Raw database files do not expose protected values.

Validation:

Obtain a database backup using the defined attacker scenario
and verify whether protected values can be recovered.

This creates an important distinction:

Implemented

≠

Effective

11. UNKNOWN becomes a legitimate result

SRIM should explicitly permit:

PASS

FAIL

UNKNOWN / NOT ESTABLISHED

Example:

"The system encrypts sensitive data."

But we don't know:

- where keys are stored,
- whether backups are encrypted,
- who can access the keys,
- what attacker is being considered.

The correct result is not necessarily:

FAIL.

It may be:

NOT ESTABLISHED.

This prevents SRIM from confusing:

lack of evidence

with

evidence of failure.

12. Reasoning Integrity vs Decision Appropriateness

This distinction must be formally preserved.

Consider:

A company chooses a relatively expensive security control because management has very low risk tolerance.

The control may be economically inefficient.

But the reasoning could still be internally coherent.

Therefore:

SRIM evaluates

Is the security reasoning supported and internally consistent?

SRIM does not automatically evaluate

Was this the best business decision?

Decision appropriateness becomes a separate dimension.

13. The SRIM evaluation dimensions

The earlier rubric can now be formally grouped.

A. Claim integrity

1. Claim boundedness
2. Threat alignment
3. Security objective

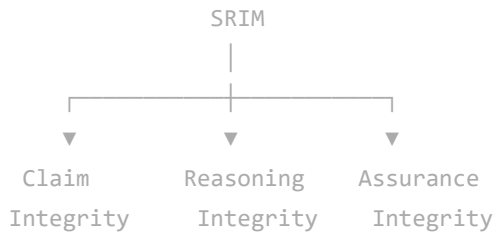
B. Reasoning integrity

4. Assumption consistency
5. Evidence sufficiency/linkage
6. Control contribution
7. Expected effect

C. Assurance integrity

8. Validation adequacy
9. Residual risk recognition
10. Epistemic honesty

This gives us a cleaner structure:



14. A preliminary formal representation

We can represent a security decision as:

$$SR = (T, O, C, A, E, K, X, V, R)$$

where:

- T = Threat
- O = Security Objective
- C = Claim
- A = Assumptions
- E = Evidence
- K = Control
- X = Expected Effect
- V = Validation
- R = Residual Risk/Uncertainty

The relationships between these elements matter more than simply having all nine fields.

15. The critical insight: SRIM is relational

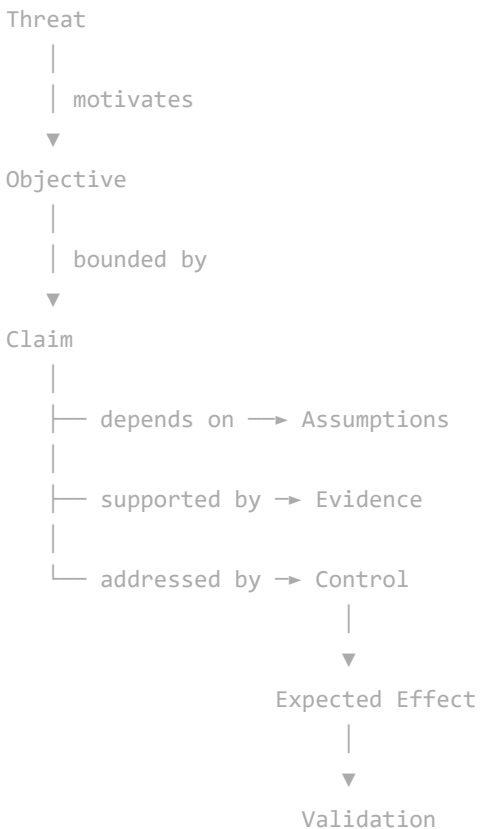
This is probably the most important change from our earliest version.

SRIM is **not**:

- ☐ Threat
- ☐ Evidence
- ☐ Control
- ☐ Validation

That would simply become another checklist.

Instead:



The edges are the important part.

16. Example

Let's apply the model.

Threat

Unauthorized access to customer records through compromised credentials.

Objective

Prevent unauthorized users from accessing records.

Claim

Role-based authorization prevents a user from accessing records outside their assigned organization.

Assumption

The authorization service receives a trustworthy user identity and organization identifier.

Evidence

Authorization tests demonstrate that users cannot retrieve records belonging to another organization.

Control

Server-side organization-level authorization checks.

Expected effect

Requests for records outside the authenticated user's organization are rejected.

Validation

Automated authorization tests plus adversarial cross-organization access testing.

Residual uncertainty

Authorization behavior of a newly introduced API endpoint has not yet been independently tested.

Now the reasoning is traceable.

17. What would constitute a reasoning failure?

We can define failure patterns independently of specific technologies.

Type 1 — Threat mismatch

The control does not address the stated threat.

Type 2 — Assumption contradiction

The claim depends on an assumption contradicted by known conditions.

Type 3 — Evidence overreach

Evidence supports a narrower claim than the claim being made.

Type 4 — Control-effect gap

A control is named, but no defensible mechanism connects it to the expected effect.

Type 5 — Validation mismatch

The validation does not test the claimed security property.

Type 6 — Unacknowledged uncertainty

Important unknowns are presented as established facts.

Type 7 — Residual-risk omission

The reasoning treats partial mitigation as elimination of the threat.

These become potential **reasoning-error classes** for the experiment.

18. What SRIM does NOT require

This is equally important.

SRIM should not require:

- a particular programming language;
- a particular cloud provider;
- a particular security product;
- a particular architecture;
- a particular framework;
- a particular encryption algorithm;
- a particular development methodology.

Otherwise SRIM becomes technology-prescriptive.

Its question is:

Does the reasoning establish an adequate relationship between the security problem, proposed intervention, evidence, and validation?

19. The independence test

Now we can return to one of your original concerns:

Could another organization implement SADF without copying our exact software?

For the **model**, the answer should ideally be yes.

If another organization can independently implement:

Threat
Objective
Claim
Assumptions
Evidence
Control
Effect
Validation
Residual uncertainty

and arrive at the same conceptual evaluation process, then the contribution is not the software itself.

The contribution is the:

method / model / formalized reasoning structure.

That is much stronger academically than making the contribution dependent on a particular application.

20. But this creates a new challenge

We have now defined SRIM.

The next question is much harder:

Are these nine entities and their relationships actually necessary?

Maybe some are redundant.

Maybe:

Objective + Claim

should be one entity.

Maybe:

Expected Effect + Validation

should be combined.

Maybe:

Residual Risk

belongs outside the reasoning model.

We should **not protect the current structure just because we created it.**

EXP-017 Verdict

SRIM v1.0 can now be described as a **relational security-reasoning model**, rather than a checklist.

Its central proposition is:

Security reasoning should make explicit and traceable the relationships between a threat, security objective, claim, assumptions, evidence, control, expected effect, validation, and residual uncertainty.

But this is still a **candidate model**, not a validated model.

The next loop should attack its architecture itself.

Loop 21 — EXP-018: Remove Parts of SRIM

We deliberately try to eliminate entities and relationships:

What is the smallest model that still preserves the claimed reasoning benefit?

That is where we can discover whether SRIM is genuinely a model—or simply a complicated checklist.

If you want, I can:

- Develop detailed SRIM v1.0 formal specification document
- Design experiments to test SRIM v1.0 necessity of all entities
- Plan iterative refinement process for SRIM v1.0 model validation



ChatGPT is AI and can make mistakes.